

Introduction to Large Language Models

Procheta Sen



What is a Language Model?

- ▶ The classic definition of a language model (LM) is a probability distribution over sequences of tokens.
- ▶ Intuitively, a good language model should have linguistic capabilities and world knowledge.
- ▶ Formally a language model defines:

$$p(x_1, x_2, \dots, x_T)$$

Using the chain rule of probability, this can be decomposed as:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1})$$

Fundamental Intuition of Large Language Models

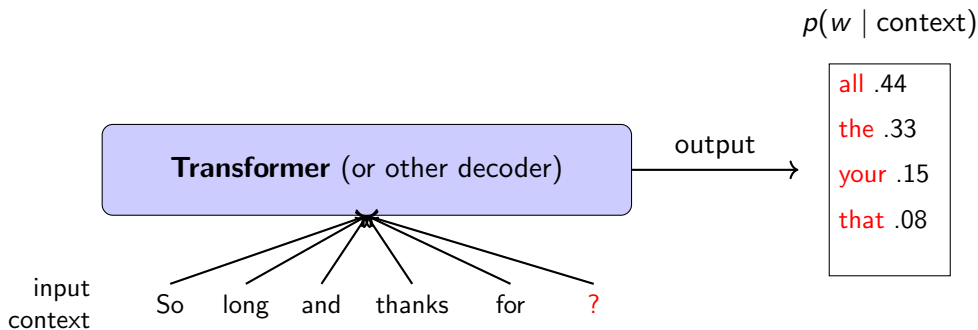
Text contains enormous amounts of knowledge.

Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much.

What is a Large Language Model?

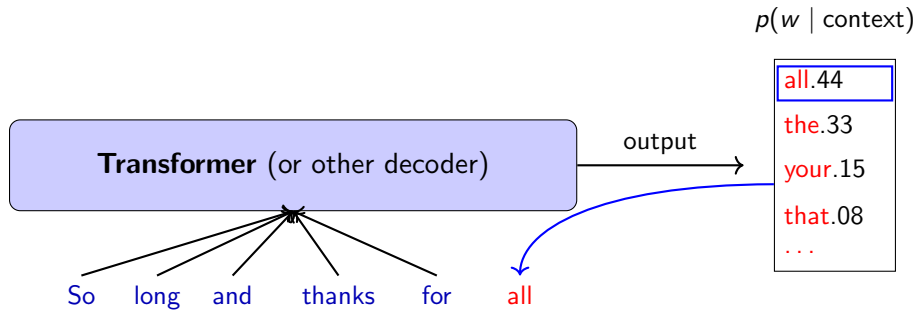
A neural network with:

- ▶ **Input:** a context or prefix
- ▶ **Output:** a distribution over possible next words



LLMs can generate!

A model that gives a probability distribution over next words can generate by repeatedly sampling from the distribution, also known as autoregressive generation.



Big idea

Many tasks can be turned into tasks of predicting words!

Conditional Generation

1. Give the LLM an input piece of text, a **prompt**
2. Have it generate token by token
 - conditioned on the prompt and the generated tokens

We generate from a model by

1. computing the probability of the next token w_i from the prior context: $P(w_i \mid w_{<i})$
2. sampling from that distribution to generate a token

NLP Tasks as Conditional Generation

Many practical NLP tasks can be cast as conditional generation!

Sentiment analysis: “I like Jackie Chan”

1. We give the language model this string:

The sentiment of the sentence ‘‘I like Jackie Chan’’ is:

2. And see what word it thinks comes next

Prompt

Prompt: a text string that a user issues to a language model to get the model to do something useful by conditional generation

Prompt engineering: the process of finding effective prompts for a task.

Prompts

A question:

What is a transformer network?

Perhaps structured:

Q: What is a transformer network? A:

Or an instruction:

Translate the following sentence into Hindi:
'Chop the garlic finely'.

Prompt Structure

Prompts can be very structured

Human: Do you think that “input” has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (

Prompt Structure

Prompts can have **demonstrations** (= examples)

The following are multiple choice questions about high school computer science.

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log n)$

Answer: C

What is the output of the statement “a + ab” in Python 3?

(A) Error (B) aab (C) ab (D) a ab

Answer:

Prompts are a learning signal

This is especially clear with demonstrations

But this is a different kind of learning than pretraining

- ▶ Pretraining sets language model weights via gradient descent
- ▶ Prompting just changes the context and the activations in the network; no parameters change

We call this **in-context learning**— learning that improves model performance but does not update parameters

LLMs usually have a system prompt

<system> You are a helpful and knowledgeable assistant.
Answer concisely and correctly.

This is automatically and silently concatenated to a user prompt

<system> You are a helpful and knowledgeable assistant.
Answer concisely and correctly. <user> **What is the capital
of France?**

System prompts can be long; 1700 words for Claude Opus4

Some extracts:

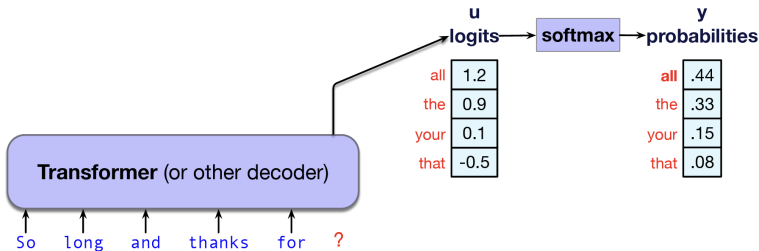
- ▶ Claude should give concise responses to very simple questions, but provide thorough responses to complex and open-ended questions.
- ▶ Claude is able to explain difficult concepts or ideas clearly. It can also illustrate its explanations with examples, thought experiments, or metaphors.
- ▶ Claude does not provide information that could be used to make chemical or biological or nuclear weapons.
- ▶ For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic.
- ▶ Claude cares about people's well-being and avoids encouraging or facilitating self-destructive behavior.
- ▶ If Claude provides bullet points in its response, it should use markdown, and each bullet point should be at least 1–2 sentences long unless the human requests otherwise.

Where does token probability come from?

The internal networks for LLMs generate real-valued scores called **logits** for each token in the vocabulary.

A score vector **u** of shape $[1 \times |V|]$ is turned into a probability by softmax:

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$



Decoding

This task of choosing a word to generate based on the model's probabilities is called **decoding**.

Decoding from a model left-to-right and repeatedly choosing the next token conditioned on our previous choices is called **autoregressive generation**.

Greedy Decoding

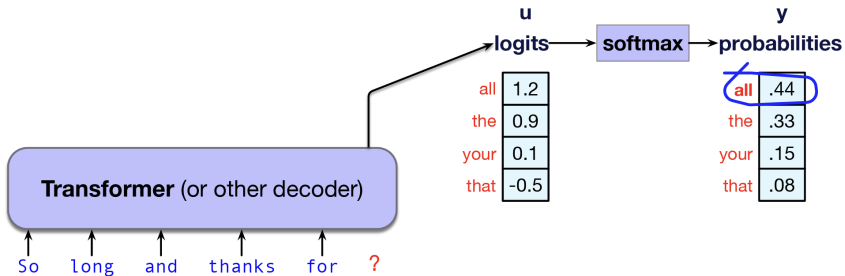
A **greedy algorithm** is one that makes a choice that is locally optimal

- (whether or not it will turn out to have been the best choice with hindsight)

Simply generate the most probable word:

$$\hat{w}_t = \arg \max_{w \in V} P(w \mid \mathbf{w}_{<t})$$

Greedy decoding: choosing “all”



We don't use greedy decoding

Because the tokens it chooses are (by definition) **extremely predictable**, the resulting text is **generic** and **repetitive**.

Greedy decoding is so predictable that it is **deterministic**.

Instead, people prefer text that is more diverse, like that generated by **sampling**.

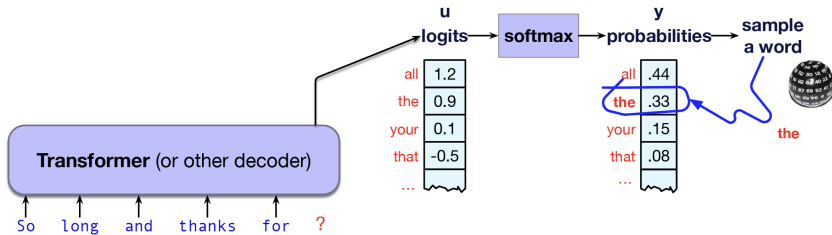
Random sampling

Sampling from a distribution means to choose random points according to their likelihood.

Sampling from an LM means to choose the next token to generate according to its probability.

Random (multinomial) sampling: We randomly select a token to generate according to its probability defined by the LM, conditioned on our previous choices, generate it, and iterate.

Random Sampling: Example



Random sampling

$$i \leftarrow 1$$

$$w_i \sim p(w)$$

while $w_i \neq \text{EOS}$

$$i \leftarrow i + 1$$

$$w_i \sim p(w_i \mid w_{<i})$$

Alas, random sampling doesn't work very well

- ▶ Even though random sampling mostly generates sensible, high-probability words,
- ▶ There are many odd, low-probability words in the **tail** of the distribution
- ▶ Each one is low-probability, but added up they constitute a large portion of the distribution
- ▶ So they get picked often enough to generate **weird sentences**

Factors in word sampling: quality and diversity

Emphasize high-probability words

- ▶ + **quality**: more accurate, coherent, and factual
- ▶ – **diversity**: boring, repetitive

Emphasize middle-probability words

- ▶ + **diversity**: more creative, diverse
- ▶ – **quality**: less factual, incoherent

Temperature sampling

Reshape the probability distribution

- ▶ increase the probability of the high-probability tokens
- ▶ decrease the probability of the low-probability tokens

Temperature Sampling

Divide the logit by a temperature parameter τ before passing it through the softmax.

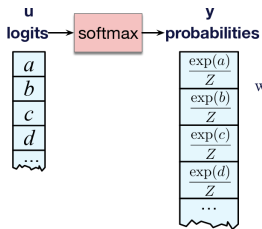
Instead of

$$\mathbf{y} = \text{softmax}(u)$$

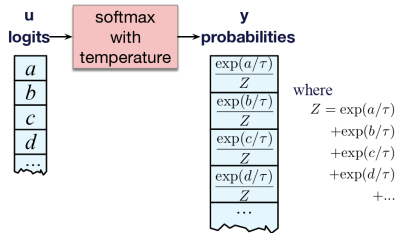
We do

$$\mathbf{y} = \text{softmax}\left(\frac{u}{\tau}\right)$$

Temperature Sampling Example



(a)



(b)

Temperature Sampling

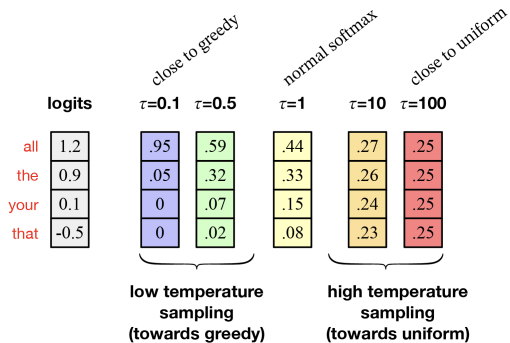
$$0 \leq \tau \leq 1$$

$$\mathbf{y} = \text{softmax}\left(\frac{\mathbf{u}}{\tau}\right)$$

Why does this work?

- ▶ When τ is close to 1, the distribution doesn't change much.
- ▶ The lower τ is, the larger the scores being passed to the softmax.
- ▶ Softmax pushes high values toward 1 and low values toward 0.
- ▶ Large inputs push high-probability words higher and low-probability words lower, making the distribution more greedy.
- ▶ As $\tau \rightarrow 0$, the probability of the most likely word approaches 1.

Softmax Output with Temperature τ



Temperature sampling comes from thermodynamics

- ▶ a system at high temperature is flexible and can explore many possible states,
- ▶ a system at lower temperature is likely to explore a subset of lower energy (better) states.

In low-temperature sampling, ($\tau \leq 1$) we smoothly

- ▶ increase the probability of the most probable words
- ▶ decrease the probability of the rare words.